

3. Introduction to Finite Volume Methods: Steady Heat Conduction

3.1	Cartesian finite-volume grids	25
3.2	Spatial discretization	25
3.3	Boundary conditions	29
3.4	The global matrix	30
3.5	Coding and verifying a simple solver	32
3.6	Guidelines for finite-volume approximations	41
3.7	Summary of key concepts	44

Introduction

This chapter introduces finite-volume methods for the case of steady heat conduction governed by the following diffusion equation,

$$\int_{CS} k(\mathbf{x})(\nabla T \cdot \mathbf{n}) dA + \int_{CV} q(\mathbf{x}) dV = 0 \quad (3.1)$$

where T is the temperature, k is the thermal conductivity, and q is a volumetric heat source. We assume $k(\mathbf{x})$ and $q(\mathbf{x})$ vary with position but not temperature. The diffusion equation (3.1) produces smooth solutions, and allows us to focus on the fundamentals of finite-volume discretization without the added complexity of advection or unsteady accumulation.

We solve Eq. (3.1) in the rectangular domain sketched in Fig. 3.1. For generality, we consider the following Robin boundary conditions,

$$\left[\alpha_w T + \beta_w \frac{\partial T}{\partial x} \right]_{x=0} = g_w, \quad \left[\alpha_e T + \beta_e \frac{\partial T}{\partial x} \right]_{x=L_x} = g_e, \quad (3.2)$$

$$\left[\alpha_s T + \beta_s \frac{\partial T}{\partial y} \right]_{y=0} = g_s, \quad \left[\alpha_n T + \beta_n \frac{\partial T}{\partial y} \right]_{y=L_y} = g_n, \quad (3.3)$$

where α , β , and g are constants, and the subscripts e , n , w , and s refer to quantities defined on the boundaries labelled *east*, *north*, *west*, and *south* in Fig. 3.1.

Note that our presentation of material in this chapter draws heavily from Patankar[1], Ferziger and Perić [2], and Moukalled, Mangani, and Darwish [3].

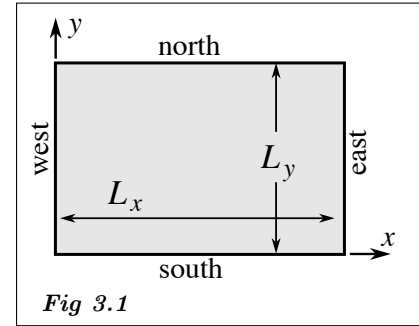


Fig 3.1

3.1 Cartesian finite-volume grids

Finite-volume methods divide the domain into small, non-overlapping control volumes called *cells*, such as that shaded gray in Fig. 3.2. The integral form of the conservation law is then applied to each cell. This textbook focuses on *Cartesian grids*, in which all cells are cuboids with faces aligned with the x -, y -, and z -axes. The grid in Fig. 3.2 is *non-uniform* because cell sizes Δx and Δy vary. When all cells are identical, the grid is called *uniform*. We define N_x and N_y as the number of cells in x - and y -directions. For the figure shown, $N_x = 4$ and $N_y = 3$.

Finite-volume methods replace continuous fields, such as $T(\mathbf{x})$, $k(\mathbf{x})$, and $q(\mathbf{x})$, with values at discrete locations. This process is called *spatial discretization*. In this book, we focus on *cell-centered* methods, where all field values are stored at cell centroids, shown as solid dots in Fig. 3.2. These points are called *nodes* or *grid points*.

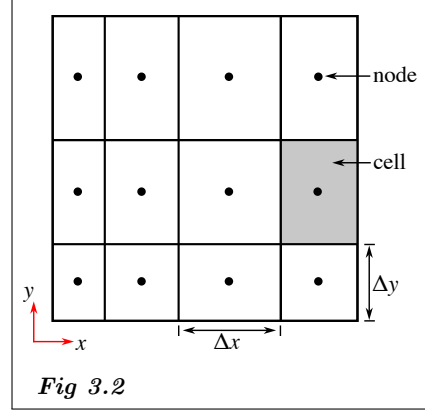


Fig 3.2

3.2 Spatial discretization

Finite-volume methods approximate conservation equations in two main steps. First, integrals are evaluated using a quadrature rule. This produces a discretized equation that depends on variables evaluated at the cell centroids and cell faces. In the second step, the face values are approximated using interpolation and finite differences. These steps are detailed in the following subsections.

3.2.1 Approximating integrals

Consider the cell shaded gray in Fig. 3.3. Though we consider 2D diffusion in the x - y plane, all cells are defined to have depth Δz in the z -direction. The solid black dots labeled P , E , W , N , and S are cell centroids. The locations marked with a blue $\times$ and labeled e , n , w , and s are the area centroids of the east, north, west, and south faces. It can be helpful to visualize these in 3D, as shown in Fig. 3.3 for s .

Because node P is located at the centroid of the control volume, we approximate the volume integral in Eq. (3.1) using the midpoint rule, which is second-order accurate,

$$\int_{CV} q(\mathbf{x}) dV \approx q_P \mathcal{V}. \quad (3.4)$$

Here $\mathcal{V} = \Delta x \Delta y \Delta z$ is the cell volume, and the subscript P denotes evaluation at the cell centroid, $q_P = q(x_P, y_P)$.

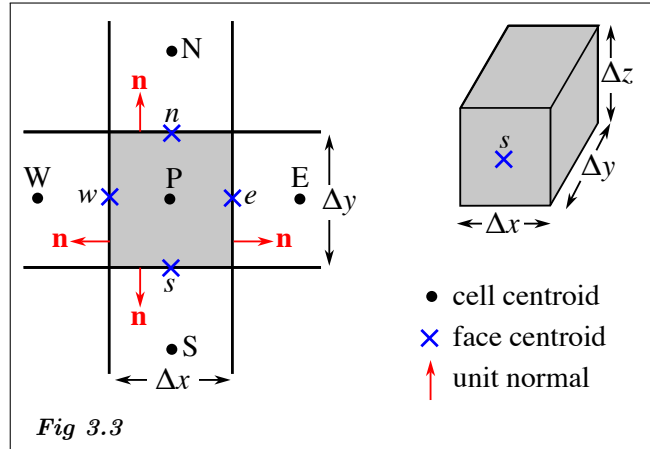


Fig 3.3

Next, we decompose the surface integral in Eq. (3.1) into four contributions over the east, north, west, and south faces,

$$\int_{CS} k(\nabla T \cdot \mathbf{n}) dA = \int_{A_e} k \frac{\partial T}{\partial x} dA + \int_{A_n} k \frac{\partial T}{\partial y} dA - \int_{A_w} k \frac{\partial T}{\partial x} dA - \int_{A_s} k \frac{\partial T}{\partial y} dA, \quad (3.5)$$

where A_e , A_n , A_w , and A_s are the face areas. The minus signs appear on the west and south faces because the outward normal $\mathbf{n}$ points in the negative x - and y -directions, respectively. Using the midpoint rule, the integrals in Eq. (3.5) are approximated to second-order accuracy as

$$\begin{aligned} \int_{CS} k(\nabla T \cdot \mathbf{n}) dA \approx & k_e \frac{\partial T}{\partial x} \Big|_e \Delta y \Delta z + k_n \frac{\partial T}{\partial y} \Big|_n \Delta x \Delta z \\ & - k_w \frac{\partial T}{\partial x} \Big|_w \Delta y \Delta z - k_s \frac{\partial T}{\partial y} \Big|_s \Delta x \Delta z, \end{aligned} \quad (3.6)$$

where the subscripts e , n , w , and s denote evaluation at face centroids.

In CFD literature [4, 3, 5], the finite-volume approximation of the diffusion term in Eq. (3.6) is often written in the compact form

$$\int_{CS} (k \nabla T \cdot \mathbf{n}) dA \approx \sum_f (k \nabla T)_f \cdot \mathbf{S}_f, \quad (3.7)$$

where $\sum_f$ denotes summation over cell faces, $(\cdot)_f$ indicates evaluation at the face centroid, and $\mathbf{S}_f = A_f \mathbf{n}$ is the *face area vector*.

This notation is especially useful for non-Cartesian grids, where cells may have an arbitrary number of faces. It allows the discretized diffusion equation to be written compactly as

$$\underbrace{\sum_f (k \nabla T)_f \cdot \mathbf{S}_f}_{\text{diffusion term}} + \underbrace{\mathcal{V} q_P}_{\text{source term}} = 0. \quad (3.8)$$

This form emphasizes that the source term scales with the cell volume $\mathcal{V}$ and depends on values at the centroid P , whereas the diffusion term depends on the face area vectors $\mathbf{S}_f$ and on field values evaluated at the face centers (e , n , w , s).

3.2.2 Approximating face values

If T and k are only known at the cell centroids, we must approximate the face values in Eq. (3.6). On Cartesian grids, the face-normal temperature gradients are typically approximated using centered differences,

$$\left. \frac{\partial T}{\partial x} \right|_e \approx \frac{T_E - T_P}{x_E - x_P}, \quad \left. \frac{\partial T}{\partial x} \right|_w \approx \frac{T_P - T_W}{x_P - x_W}, \quad (3.9)$$

$$\left. \frac{\partial T}{\partial y} \right|_n \approx \frac{T_N - T_P}{y_N - y_P}, \quad \left. \frac{\partial T}{\partial y} \right|_s \approx \frac{T_P - T_S}{y_P - y_S}. \quad (3.10)$$

On uniform grids, these approximations are formally second-order accurate. On nonuniform grids, formal second-order accuracy is lost, but is often observed if the cell sizes Δx and Δy do not vary too rapidly.¹

On Cartesian grids, face values of the thermal conductivity k are typically obtained by interpolation between adjacent cell centroids. For smoothly varying $k(\mathbf{x})$, linear interpolation is usually chosen. For example, the eastern face value k_e in Fig. 3.3 can be obtained by assuming k varies linearly between P and E ,

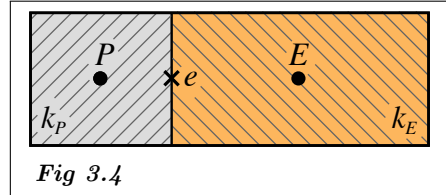
$$\frac{k_E - k_P}{x_E - x_P} = \frac{k_e - k_P}{x_e - x_P}, \quad (3.11)$$

which gives

$$k_e = \sigma k_P + (1 - \sigma)k_E, \quad \text{where} \quad \sigma = \frac{x_E - x_e}{x_E - x_P}. \quad (3.12)$$

3.2.3 Harmonic mean interpolation

Linear interpolation of k is appropriate when the thermal conductivity varies smoothly. However, in cases where k varies rapidly, or is discontinuous at the interface between two materials, an alternate interpolation is often preferred [1]. To explain why, consider the one-dimensional steady heat conduction between the two cells P and E in Fig. 3.4. The material in cell P has the conductivity k_P , while that in cell E has the conductivity k_E . The two materials meet at the shared face labeled e .



If we require the heat flux q_e to be continuous across the face, then

$$q_e = -k_P \left(\frac{T_e - T_P}{x_e - x_P} \right) = -k_E \left(\frac{T_E - T_e}{x_E - x_e} \right), \quad (3.13)$$

where T_e is the (unknown) temperature at the face. Solving for T_e and substituting back, the heat flux can be written as

$$q_e = -k_e \frac{T_E - T_P}{x_E - x_P}, \quad (3.14)$$

where the effective face conductivity k_e is the harmonic mean,

$$k_e = \left(\frac{1 - \sigma}{k_P} + \frac{\sigma}{k_E} \right)^{-1}, \quad \sigma = \frac{x_e - x_P}{x_E - x_P}. \quad (3.15)$$

¹For a detailed discussion, see Ferziger and Perić [2].

On a uniform grid with x_e midway between P and E , this reduces to the harmonic mean,

$$k_e = \frac{2k_P k_E}{k_P + k_E}. \quad (3.16)$$

Patankar [1] interprets Eq. (3.15) by considering the following limiting cases:

- In the limits $k_E \rightarrow 0$ or $k_P \rightarrow 0$, Eq. (3.15) produces $k_e = 0$, and the cell face acts as an insulator, as it should. In contrast, linear interpolation would produce a non-zero k_e and therefore a non-zero heat flux.
- In the limit $k_P \gg k_E$, the term $(1 - \sigma)/k_P$ in Eq. (3.15) becomes negligible, giving $k_e \approx k_E/\sigma$. This is physically consistent with the fact that when $k_P \gg k_E$, the temperature drop between P and e is negligible, and the heat flux is controlled entirely by the temperature drop between T_e and T_E in the low-conductivity cell E,

$$q_e \approx -k_E \left(\frac{T_E - T_P}{x_E - x_e} \right) = -\frac{k_E}{\sigma} \left(\frac{T_E - T_P}{x_E - x_P} \right), \quad (3.17)$$

consistent with $k_e \approx k_E/\sigma$.

3.2.4 The 5-point stencil

Using centered differences for the face-normal heat fluxes $(\mathbf{n} \cdot \nabla T)_f$ and linear interpolation for the face thermal conductivities k_f , the finite-volume approximation of the steady heat conduction equation can be written as the algebraic equation below [1].

The discretized steady, 2D, heat conduction equation:

$$a_P T_P = a_E T_E + a_N T_N + a_W T_W + a_S T_S + b, \quad (3.18)$$

where

$$a_E = \frac{k_e \Delta y}{x_E - x_P}, \quad a_W = \frac{k_w \Delta y}{x_P - x_W}, \quad (3.19)$$

$$a_N = \frac{k_n \Delta x}{y_N - y_P}, \quad a_S = \frac{k_s \Delta x}{y_P - y_S}, \quad (3.20)$$

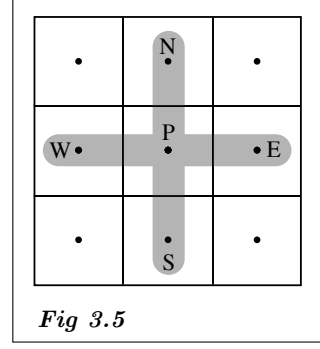
$$a_P = a_E + a_N + a_W + a_S, \quad b = q_P \Delta x \Delta y. \quad (3.21)$$

Equation (3.18) couples the temperature T_P at the centroid of each cell to the temperatures in the four adjacent cells, as sketched in Fig. 3.5. This coupling is often called the *5-point stencil*, or the *7-point stencil* for 3D Cartesian grids. For the discussion in the upcoming sections, it is convenient to express Eq. (3.18) using the notation of Patankar [1],

$$a_P T_P = \sum_{nb} a_{nb} T_{nb} + b, \quad (3.22)$$

where $\sum_{nb}$ denotes summation over the four neighboring cells,

$$\sum_{nb} a_{nb} c_{nb} = a_E T_E + a_N T_N + a_W T_W + a_S T_S. \quad (3.23)$$



3.3 Boundary conditions

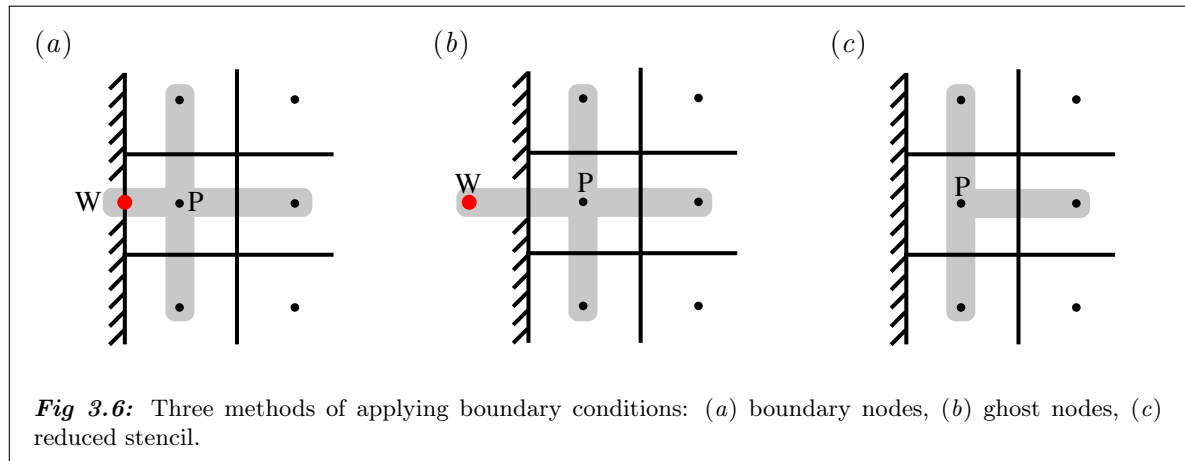
When a cell has one or more faces on the domain boundary, the standard 5-point stencil is incomplete, because one or more adjacent cells do not exist. Boundary conditions must therefore be incorporated into the discrete equations. Here we present three methods that account for boundary conditions by modifying the stencil or introducing additional grid points.

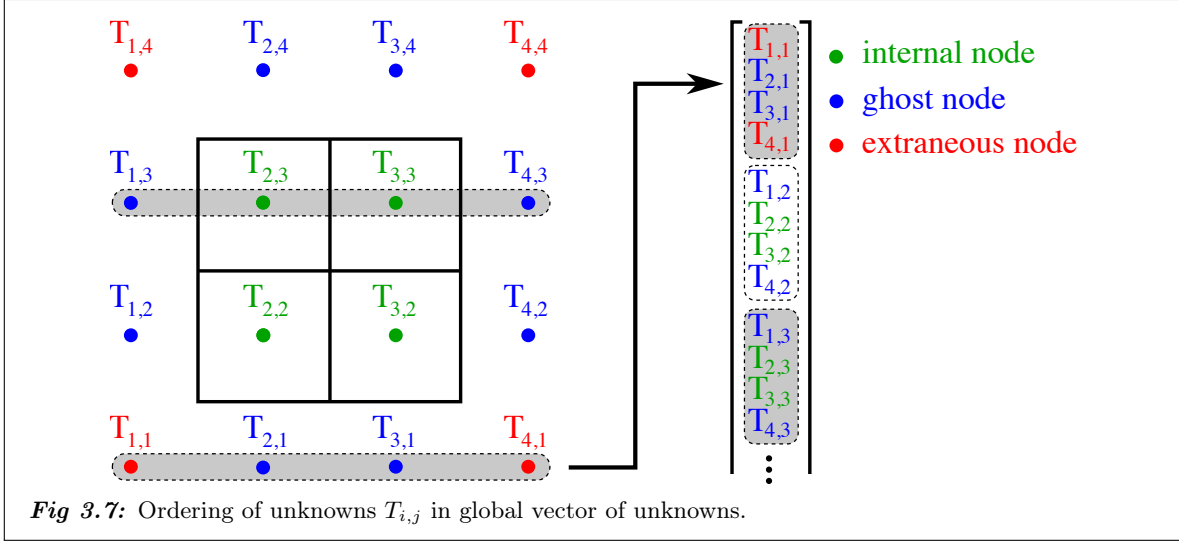
3.3.1 Boundary nodes

Figure 3.6(a) illustrates a cell with centroid P and a west face on the boundary. The first approach introduces a boundary node at the centroid of the boundary face. Using this node, the 5-point stencil is modified to account for the new location of x_W , but otherwise remains similar. The Robin condition is then applied by approximating $\partial T / \partial x$ at the boundary face with a one-sided difference. The main drawback is that achieving second-order accuracy requires a more complicated one-sided approximation involving three grid points.

3.3.2 Ghost nodes

An alternate approach, illustrated in Fig. 3.6(b), introduces a *ghost node* T_W outside the domain so that the boundary face lies midway between the ghost node and centroid P . This





allows the usual 5-point stencil to be applied without modification. Furthermore, the Robin condition can be approximated to second-order spatial accuracy as

$$\alpha_w \left(\frac{T_P + T_W}{2} \right) + \beta_w \left(\frac{T_P - T_W}{\Delta x} \right) = g_w, \quad (3.24)$$

where Δx is the distance between P and the ghost node. We can express this approximation as

$$a_P T_P + a_W T_W = g_w, \quad (3.25)$$

where

$$a_P = \frac{\alpha_w}{2} + \frac{\beta_w}{\Delta x}, \quad a_W = \frac{\alpha_w}{2} - \frac{\beta_w}{\Delta x}. \quad (3.26)$$

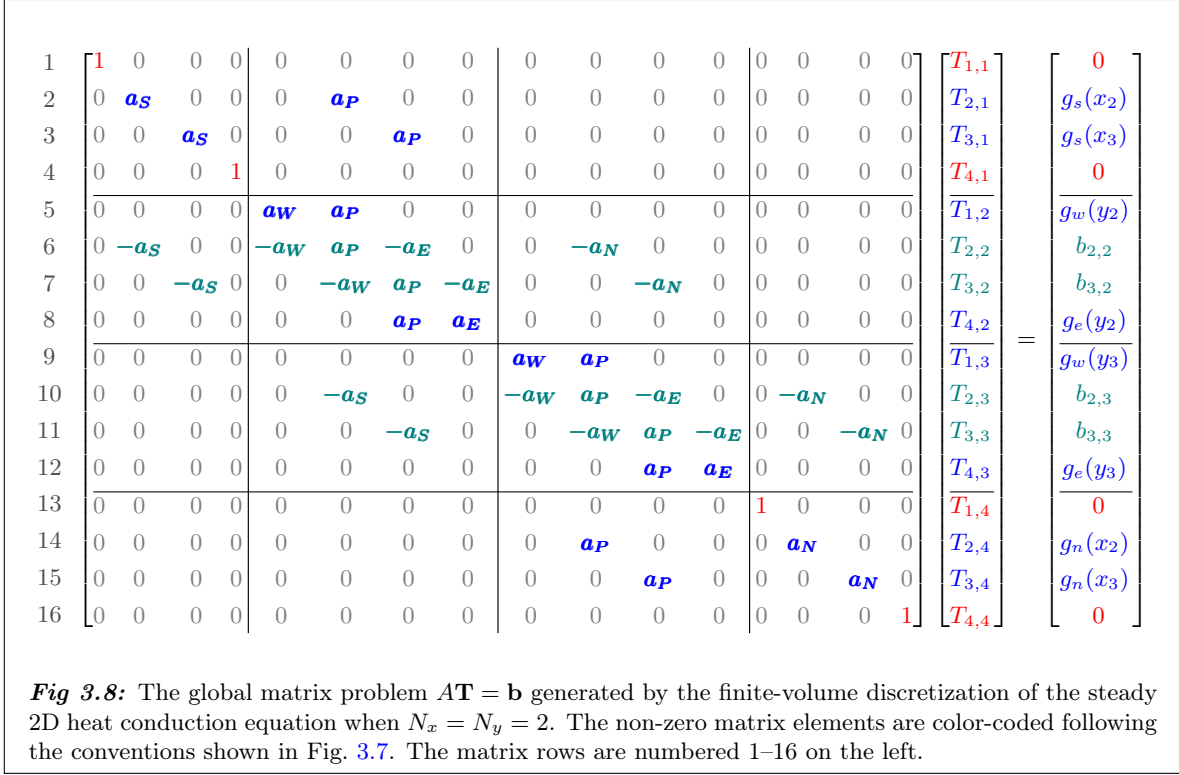
For the scope of this introductory textbook, we apply boundary conditions using ghost nodes, because they allow the usual 5-point stencil to be applied on all interior cells, which simplifies coding. The disadvantage is that ghost nodes introduce extra unknowns at the corners of the domain, which require special treatment, as discussed later in Section 3.4.

3.3.3 Reduced boundary stencils

The third method avoids extra nodes by solving Eq. (3.25) for T_W and substituting it into the 5-point stencil, reducing it to four points, as in Fig. 3.6(c). This removes extraneous nodes and reduces the number of unknowns, but adds coding overhead. Of the three methods above, it is arguably the cleanest and extends naturally to unstructured grids. For that reason, it is used by CFD software such as Fluent and OpenFOAM. However, for the scope of this introductory textbook, we use ghost nodes because they are simpler to implement.

3.4 The global matrix

Figure 3.7 shows a finite-volume grid with four cells ($N_x = N_y = 2$). Due to the added ghost nodes, the computational grid contains $4 \times 4 = 16$ nodal temperatures. To solve for these 16 unknowns, we require 16 linearly independent equations. On the four interior nodes



shaded green, we apply the discrete heat conduction equation (3.18). On the eight ghost nodes shaded blue, we apply the discrete Robin conditions, using the approach shown in Eq. (3.25). There remain, however, four additional ghost nodes shaded red in Fig. 3.7. The temperatures at these nodes have no impact on the numerical solution, because they are never accessed by the discretizations of the PDE and boundary conditions. Though we could remove these extraneous nodes from the grid, doing so would complicate the indexing, as we show shortly. We therefore retain them and arbitrarily set $T = 0$ at these locations.

To solve the resulting system of 16 equations, we express it as a *global matrix problem* of the form

$$\mathbf{AT} = \mathbf{b}, \quad (3.27)$$

where $\mathbf{A}$ is a 16×16 coefficient matrix, $\mathbf{T}$ is a 16×1 column vector of unknown temperatures, and $\mathbf{b}$ is a 16×1 column vector containing the source terms. These include terms such as $b = q_P \Delta x \Delta y$ from the discrete heat conduction equation (3.21), and the g_w term from the discrete boundary condition (3.25).

3.4.1 Ordering unknowns and assembling matrix rows

To assemble Eq. (3.27), we must choose an ordering for the unknown temperatures in the vector $\mathbf{T}$. For that, we first index the 16 unknowns as $T_{i,j}$, where $i \in [1, 4]$ denotes the x -index and $j \in [1, 4]$ denotes the y -index. Using these indices, we order $\mathbf{T}$ starting with $T_{1,1}$ and then proceed rightward along each row of the grid, as illustrated in Fig. 3.7. This convention produces the matrix system shown in Fig. 3.8. To help interpret the figure, the non-zero elements of $\mathbf{A}$ are shaded green, blue, or red, depending on whether they are generated by the discretized PDE, boundary conditions, or extraneous ghost nodes, respectively, using the

same color scheme as in Fig. 3.7. We note the following:

- The 5-point stencil generated by the discretized heat conduction equation occurs on rows 6, 7, 10, and 11.
- The temperature at the four extraneous nodes is set to zero on rows 1, 4, 13, and 16.
- The remaining rows apply boundary conditions. For example, rows 2–3 apply the condition on the south boundary ($y = 0$), rows 14–15 apply the condition on the north boundary $y = L_y$, rows 5 and 9 apply the conditions on the west boundary ($x = 0$), and rows 8 and 12 apply the conditions on the east boundary ($x = L_x$).
- On rows applying boundary conditions, there are typically two nonzero entries. The diagonal entry corresponds to the ghost node (a_W , a_E , a_S , or a_N depending on the boundary), and the off-diagonal entry contains a_P from the adjacent interior node.

3.5 Coding and verifying a simple solver

In this section, we demonstrate the coding and verification of a finite-volume approximation of the steady 2D heat conduction equation with constant conductivity k ,

$$k\nabla^2 T = -q(x, y). \quad (3.28)$$

We verify our code using the *method of manufactured solutions*. In that approach, we first choose the “manufactured” test solution

$$T_e(x, y) = \cos(x) \cos(y). \quad (3.29)$$

We then substitute the solution in Eq. (3.28) and solve for the required source $q(x, y)$,

$$q(x, y) = -k\nabla^2 T_e = 2k \cos(x) \cos(y). \quad (3.30)$$

We solve the heat conduction equation in the rectangular domain introduced previously in Fig. 3.1, using a uniform grid with the constant cell dimensions

$$\Delta x = \frac{L_x}{N_x}, \quad \Delta y = \frac{L_y}{N_y}. \quad (3.31)$$

We apply the following Dirichlet boundary conditions, which are chosen to satisfy the manufactured solution (3.29),

$$T|_{x=0} = \cos(y), \quad T|_{x=L_x} = \cos(L_x) \cos(y), \quad (3.32)$$

$$T|_{y=0} = \cos(x), \quad T|_{y=L_y} = \cos(x) \cos(L_y). \quad (3.33)$$

In the main Matlab routine shown in File 3.1 below, we organize our code in five main steps: (1) building the grid, (2) assembling the coefficient matrix $\mathbf{A}$, (3) constructing the right-hand side vector $\mathbf{b}$, (4) solving the linear system, and (5) post-processing the results. The following subsections examine each step in detail.

File 3.1: The main Matlab file Main.m

```
1 function[ERR] = Main(Lx,Ly,Nx,Ny,k,PLOTFLAG)
2
3 % (1) build grid
4 [x,y] = Grid(Lx,Ly,Nx,Ny,0);
5
6 % (2) build A
7 A = BuildA(Lx,Ly,Nx,Ny,k,0);
8
9 % (3) build right hand side
10 B = RHS(Lx,Ly,Nx,Ny,x,y,k);
11
12 % (4) solve linear system
13 TMP = A\B;
14
15 T = zeros(Nx+2,Ny+2);
16 for i=1:Nx+2
17     for j=1:Ny+2
18         row = (j-1)*(Nx+2) + i;
19         T(i,j) = TMP(row);
20     end
21 end
22
23 % (5) post-processing
24 ERR = PostProcess(Nx,Ny,x,y,T,PLOTFLAG);
```

3.5.1 Building the grid

When solving a general heat conduction problem on a nonuniform, Cartesian grid, the key quantities to compute are the coordinates of the cell centroids and face centroids, as well as the dimensions Δx and Δy of each cell. These quantities are needed to interpolate the face thermal conductivities and compute the coefficients in the global matrix. However, for our simple problem here, the cell dimensions are constant, and we do not need to interpolate k . The Matlab routine `Grid.m` shown in File (3.2) below nevertheless computes the coordinates of both the cell centroids (x,y) and face centroids (xf,yf) . It also optionally plots the grid when the input `PLOTFLAG` has the value 1. The coordinates of the cell centroids are computed on lines 8–9, where they are stored as the vectors x and y , which contain $N_x + 2$ and $N_y + 2$ cell-center locations, including one ghost cell on each side of the domain. For example, the vector x contains $N_x + 2$ uniformly distributed coordinates between $-dx/2$ and $L_x + dx/2$.

File 3.2: The Matlab routine Grid.m

```
1 function[x,y,xf,yf] = Grid(Lx,Ly,Nx,Ny,PLOTFLAG)
2
3 % --- build grid ---
4 dx = Lx/Nx;
5 dy = Ly/Ny;
6 xf = linspace(0,Lx,Nx+1);
7 yf = linspace(0,Ly,Ny+1);
8 x = linspace(-0.5*dx,Lx+0.5*dx,Nx+2);
9 y = linspace(-0.5*dy,Ly+0.5*dy,Ny+2);
10
```

```

11 % plot grid
12 if (PLOTFLAG == 1)
13
14     figure
15     set(gca, 'FontName', 'Times', 'FontSize', 20)
16     hold on
17
18     % plot cell faces
19     for i = 1:Nx+1
20         plot([xf(i) xf(i)], [0 Ly], '-k', 'LineWidth', 2.0)
21     end
22     for j = 1:Ny+1
23         plot([0 Lx], [yf(j) yf(j)], '-k', 'LineWidth', 2.0)
24     end
25
26     % plot grid points
27     for i = 1:Nx+2
28         for j = 1:Ny+2
29             plot(x(i), y(j), '.k', 'MarkerSize', 20)
30         end
31     end
32
33     % formatting
34     box on
35     grid on
36     xlabel('$x$', 'interpreter', 'latex', 'FontSize', 20)
37     ylabel('$y$', 'interpreter', 'latex', 'FontSize', 20, 'rot', 0)
38
39 end

```

3.5.2 Building the $\mathbf{A}$ matrix

When $N_x = N_y = N$, the solution vector $\mathbf{T}$ in Fig. 3.8 contains $(N + 2)^2$ unknowns, and the full matrix $\mathbf{A}$ contains $(N + 2)^4$ elements. Storing the full matrix in double precision (8 bytes per element) requires

$$\text{Mem} = \frac{8}{10^9} (N + 2)^4 \quad \text{GB.} \quad (3.34)$$

On a 32 GB machine, this limits the grid to roughly $N = 249$, and the usable limit is smaller because additional memory is needed for the solver and temporary arrays. For a 3D grid with $N_x = N_y = N_z = N$, the number of unknowns becomes $(N + 2)^3$, and storing the full matrix requires

$$\text{Mem} = \frac{8}{10^9} (N + 2)^6 \quad \text{GB,} \quad (3.35)$$

which restricts the grid to about $N < 38$.

Fortunately, these limits can be exceeded because $\mathbf{A}$ is sparse. For the 2D heat conduction equation, each row has at most five nonzero entries. Storing only those values gives

$$\text{Mem} = \frac{8}{10^9} 5(N + 2)^2 \quad \text{GB.} \quad (3.36)$$

With this storage, a grid with $N = 2048$ requires only about 0.17 GB, though this estimate does not include the memory needed to store the locations of the nonzero entries. In MATLAB,

sparse matrices are stored using the *Compressed Sparse Column* (CSC) format, which stores both indices and values efficiently and supports fast matrix operations.

In MATLAB, an empty sparse matrix of size $\text{NR} \times \text{NC}$ can be created using

```
A = sparse(NR,NC);
```

Entries can then be assigned using

```
A(i,j) = value;
```

This approach is simple but becomes slow for large problems because each insertion updates MATLAB's internal sparse structure. A more efficient strategy is to first store all nonzero entries in three arrays: **I**, **J**, and **VAL**. Arrays **I** and **J** store row and column indices, and **VAL** stores the corresponding values. The sparse matrix is then assembled using

```
A = sparse(I,J,VAL);
```

We use this approach in the routine **BuildA** shown in File 3.3 below. Throughout **BuildA.m**, nonzero matrix entries are added to the array **A** using the helper function **AddVal**, which is defined later in this section.

File 3.3: Building the **A** matrix for a uniform grid with constant thermal conductivity k and Dirichlet boundary conditions

```

1 function[As] = BuildA(Lx,Ly,Nx,Ny,k,PLOTSPY)
2
3 % --- number of nodes including ghost nodes ---
4 Nx2 = Nx + 2;
5 Ny2 = Ny + 2;
6
7 % --- allocate matrix used to store nonzeros ---
8 Nmax = Nx2*Ny2*5; % estimate max number of nonzeros
9 A = zeros(Nmax,3);
10 ctr = 0;
11
12 % grid spacing
13 dx = Lx/Nx;
14 dy = Ly/Ny;
15
16 % coefficients in 5-point stencil
17 ae = k*dy/dx;
18 aw = k*dy/dx;
19 an = k*dx/dy;
20 as = k*dx/dy;
21 ap = ae + aw + an + as;
22
23 % south/north boundary conditions
24 for i = 2:Nx+1
25
26     % south boundary
27     row = i;
28     [A,ctr] = AddVal(row, row, 0.5, A, ctr);
29     [A,ctr] = AddVal(row, row + Nx2, 0.5, A, ctr);
30
31     % north boundary
32     row = (Ny+1)*Nx2 + i;
```

```

33     [A,ctr] = AddVal(row, row, 0.5, A, ctr);
34     [A,ctr] = AddVal(row, row - Nx2, 0.5, A, ctr);
35
36 end
37
38 % west/east boundary conditions
39 for j = 2:Ny+1
40
41     % west boundary
42     row = (j-1)*Nx2 + 1;
43     [A,ctr] = AddVal(row, row, 0.5, A, ctr);
44     [A,ctr] = AddVal(row, row + 1, 0.5, A, ctr);
45
46     % east boundary
47     row = j*Nx2;
48     [A,ctr] = AddVal(row, row, 0.5, A, ctr);
49     [A,ctr] = AddVal(row, row - 1, 0.5, A, ctr);
50
51 end
52
53 % --- apply 5-point stencil on interior nodes ---
54 for i = 2:Nx+1
55     for j = 2:Ny+1
56
57         row = (j-1)*Nx2 + i;
58
59         p = row;
60         e = row + 1;
61         w = row - 1;
62         n = row + Nx2;
63         s = row - Nx2;
64
65         [A,ctr] = AddVal(row, p, ap, A, ctr);
66         [A,ctr] = AddVal(row, e, -ae, A, ctr);
67         [A,ctr] = AddVal(row, w, -aw, A, ctr);
68         [A,ctr] = AddVal(row, n, -an, A, ctr);
69         [A,ctr] = AddVal(row, s, -as, A, ctr);
70     end
71 end
72
73 % --- extraneous corner nodes --
74 [A,ctr] = AddVal(1, 1, 1, A, ctr);
75 [A,ctr] = AddVal(Nx2, Nx2, 1, A, ctr);
76 [A,ctr] = AddVal((Ny+1)*Nx2 + 1, (Ny+1)*Nx2 + 1, 1, A, ctr);
77 [A,ctr] = AddVal(Ny2*Nx2, Ny2*Nx2, 1, A, ctr);
78
79 % --- assemble sparse matrix ---
80 A = A(1:ctr,:); % remove extra entries
81 As = sparse(A(:,1), A(:,2), A(:,3));
82
83 % --- optional spy plot ---
84 if (PLOTSPY == 1)
85     figure
86     spy(As)

```

```

87     title('Sparsity pattern of A')
88 end

```

`BuildA.m` stores the matrix elements in the array `A`, in which each row represents a nonzero entry stored as the triplet `[i, j, value]`. We begin on lines 8–9 by allocating an array `A` of size $N_{\max} \times 3$, where $N_{\max} = 5(N_x + 2)(N_y + 2)$ is an estimate of the maximum number of nonzero entries. Line 10 initializes the variable `ctr = 0`, which will be incremented each time we add a new nonzero entry to `A`.

A nonzero matrix element is added to the array `A` using the syntax below,

```
[A,ctr] = AddVal(row,col,val,A,ctr);
```

where `AddVal` is the function defined below

```

1 function[A,ctr] = AddVal(row,col,val,A,ctr)
2
3 ctr = ctr + 1;
4 A(ctr,:) = [row, col, val];

```

For example, line 74 adds the element $a_{1,1} = 1$ using the command

```
74 [A,ctr] = AddVal(1, 1, 1, A, ctr);
```

Throughout the code, the 2D grid index (i, j) is mapped to the corresponding matrix row using the relation

$$\text{row} = (j - 1)(N_x + 2) + i. \quad (3.37)$$

After all nonzero entries have been assembled, lines 80–81 remove any excess rows of `A`, and save the matrix using MATLAB's internal sparse format.

3.5.3 Building the right-hand-side vector

The routine `RHS.m` in File (3.4) below assembles the right-hand-side vector `b`, using the same row-by-row indexing used to build the `A` matrix. Note that we never explicitly set entries for the extraneous ghost nodes, because line 8 initializes the vector `B` to zero.

File 3.4: Building the right-hand-side `b` vector.

```

1 function[B] = RHS(Lx,Ly,Nx,Ny,x,y,k)
2
3 % --- initialize RHS vector ---
4 Nx2 = Nx + 2;
5 Ny2 = Ny + 2;
6 B = zeros(Nx2*Ny2,1);
7
8 % --- grid spacing and cell area ---
9 dx = Lx/Nx;
10 dy = Ly/Ny;
11 dA = dx*dy;
12
13 % south/north boundary conditions
14 for i = 2:Nx+1
15     % south boundary (y = 0)
16     row = i;
17     B(row) = cos(x(i)); % Dirichlet BC: T = cos(x)
18

```

```

19     % north boundary (y = Ly)
20     row = (Ny+1)*Nx2 + i;
21     B(row) = cos(x(i))*cos(Ly); % Dirichlet BC: T = cos(x)*cos(Ly)
22 end
23
24 % west/east boundary conditions
25 for j = 2:Ny+1
26     % west boundary (x = 0)
27     row = (j-1)*Nx2 + 1;
28     B(row) = cos(y(j)); % Dirichlet BC: T = cos(y)
29
30     % east boundary (x = Lx)
31     row = j*Nx2;
32     B(row) = cos(Lx)*cos(y(j)); % Dirichlet BC: T = cos(Lx)*cos(y)
33 end
34
35 % --- interior nodes -----
36 for j = 2:Ny+1
37     for i = 2:Nx+1
38
39         row = (j-1)*Nx2 + i;
40         B(row) = 2*k*cos(x(i))*cos(y(j))*dA;
41
42     end
43 end

```

3.5.4 Solving the linear system

Lines 13–21 of Main.m, shown in File 3.5 below, solve the linear system and store the solution in the 2D array T.

File 3.5: Solving the linear system

```

13 TMP = A\B;
14
15 T = zeros(Nx+2,Ny+2);
16 for i = 1:Nx+2
17     for j = 1:Ny+2
18         row = (j-1)*(Nx+2) + i;
19         T(i,j) = TMP(row);
20     end
21 end

```

The operation $\text{TMP} = \mathbf{A} \backslash \mathbf{B}$ solves the matrix problem $\mathbf{A}\mathbf{T} = \mathbf{b}$ and stores the solution vector in the temporary array TMP. The subsequent loops map this vector back to the 2D grid, using the same row-by-row indexing relation $\text{row} = (j-1)*(Nx+2) + i$. MATLAB's backslash operator automatically detects that $\mathbf{A}$ is sparse and typically uses an efficient sparse direct solver (UMFPACK). The details of linear-system solution algorithms are discussed in a later chapter.

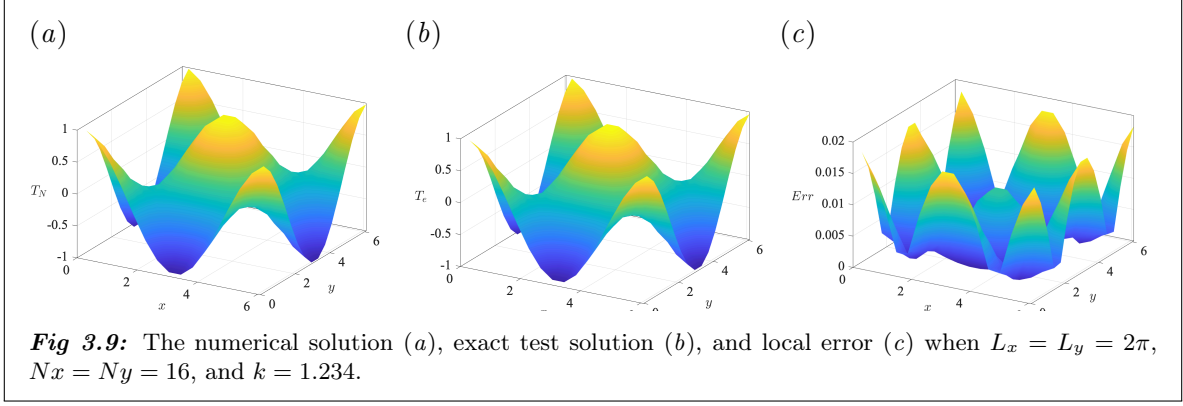
3.5.5 Postprocessing

The routine `PostProcess.m`, shown in File (3.6) below, computes the numerical error and, if requested, plots the results.

File 3.6: The `PostProcess.m` routine computes the error and plots results

```
1 function[ERR] = PostProcess(Nx,Ny,x,y,T,PLOTFLAG)
2
3 % compute test solution
4 [XP,YP] = meshgrid( x(2:Nx+1), y(2:Ny+1) );
5 Te = cos(XP).*cos(YP);
6
7 % extract numerical solution
8 TP = T(2:Nx+1,2:Ny+1);
9 TP = TP';
10
11 % compute local and global error
12 Err = abs(TP-Te);
13 ERR = max(max(abs(Err)));
14
15 % plot results
16 if (PLOTFLAG == 1)
17
18     figure
19
20     % plot numerical result
21     subplot(1,3,1)
22     surf(XP,YP,TP);
23     shading interp
24     xlabel('$x$', 'interpreter', 'latex', 'FontSize', 20)
25     ylabel('$y$', 'interpreter', 'latex', 'FontSize', 20)
26     zlabel('$T_N$', 'interpreter', 'latex', 'FontSize', 20)
27
28     % plot test solution
29     subplot(1,3,2)
30     surf(XP,YP,Te);
31     shading interp
32     xlabel('$x$', 'interpreter', 'latex', 'FontSize', 20)
33     ylabel('$y$', 'interpreter', 'latex', 'FontSize', 20)
34     zlabel('$T_e$', 'interpreter', 'latex', 'FontSize', 20)
35
36     % plot local error
37     subplot(1,3,3)
38     surf(XP,YP,Err);
39     shading interp
40     xlabel('$x$', 'interpreter', 'latex', 'FontSize', 20)
41     ylabel('$y$', 'interpreter', 'latex', 'FontSize', 20)
42     zlabel('$Err$', 'interpreter', 'latex', 'FontSize', 20)
43
44 end
```

Lines 4–5 of File (3.6) evaluate the analytical test solution $T_e(x, y)$ at the interior nodes. The command `meshgrid` constructs the coordinate matrices `XP` and `YP`, each of size $N_y \times N_x$, containing the (x, y) coordinates of all interior points. Note, however, that the indexing of



matrices created by `meshgrid` differs from the (i, j) ordering used in our solver. Specifically,

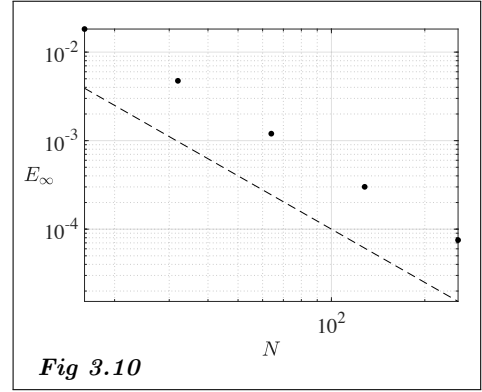
$$x(i) = XP(j, i), \quad y(j) = YP(j, i). \quad (3.38)$$

Thus, the first index corresponds to the y -direction and the second to the x -direction.

Line 8 extracts the numerical solution at the interior nodes, removing the ghost cells. Line 9 then transposes `TP` so that its indexing matches that of `XP`, `YP`, and `Te`. Lines 12–13 then compute the local error and the global error. If `PLOTFLAG` equals 1, the remaining lines generate surface plots of the numerical solution, analytical solution, and local error. These plots provide a visual check of solution smoothness and error behavior.

3.5.6 Verification

Figure 3.9 illustrates surface plots of the numerical solution (panel a), exact test solution (panel b), and local error (panel c) when $L_x = L_y = 2\pi$, $N_x = N_y = 16$, and $\Gamma = 1.234$. Figure 3.10 shows a corresponding log-log plot of the global error E_∞ vs. $N_x = N_y = N$. The dashed line shows the slope $1/N_x^2$, confirming second-order accuracy.



Practical advice when verifying codes: It is good practice to test codes using cases with $L_x \neq L_y$ and $N_x \neq N_y$, and to avoid setting physical properties to unity, such as $k = 1$. Common coding errors include the accidental use of N_x where N_y is required, the use of L_x in place of L_y , and the omission of material properties in the discretized equations. For unequal grid spacings, error plots can be constructed by refining the mesh while maintaining a constant ratio between N_x and N_y , for example, $(N_x, N_y) = (12, 16), (24, 32), (36, 48), \dots$. The manufactured solution should also reflect the complexity of the target problem. Overly simple functions, such as $T(x, y) = xy$, may fail to exercise all terms in the governing equations.

Verification, validation, and mesh independence: It should be stressed that the method of manufactured solutions *verifies* whether a numerical method approximates the underlying PDEs with the expected order of accuracy. It does not, however, assess the accuracy of a

real-world simulation. That requires additional tools that fall under the concepts of *mesh independence* and *validation*, which we discuss later. Verification is nevertheless an important tool. It is used by software developers during code development, and by end-users when unexpected behavior is observed or when the software is extended through “user-defined functions” in FLUENT or source-code modifications in OPENFOAM.

3.6 Guidelines for finite-volume approximations

It is important to understand the principles that make finite-volume methods physically reliable and numerically robust. This section summarizes several rules and guiding principles emphasized by Patankar [1] and by Moukalled et al. [3] that good finite-volume discretizations should satisfy.

3.6.1 Consistent flux approximations

A defining feature of finite-volume methods is that they enforce *discrete conservation*. When a face is shared by two neighboring control volumes, the numerical approximation of any flux through that face must ensure that the flux leaving one cell is equal and opposite to that entering the neighboring cell. By consistently approximating fluxes in this manner, finite-volume methods ensure that the integral form of a conservation law is satisfied over the entire discretized domain, even on coarse grids.

Our centered-difference approximations to $\nabla T \cdot \mathbf{n}$ in Eqs. (3.9)–(3.10) satisfy this requirement. For example, the conductive heat flux through the east face of cell P in Fig. 3.11,

$$\int_{A_e} k(\nabla T \cdot \mathbf{n}) dA \approx k_e \frac{T_E - T_P}{x_E - x_P} \Delta y \Delta z,$$

is equal and opposite to that entering the west face of cell E ,

$$\int_{A_w} k(\nabla T \cdot \mathbf{n}) dA \approx -k_e \frac{T_E - T_P}{x_E - x_P} \Delta y \Delta z.$$

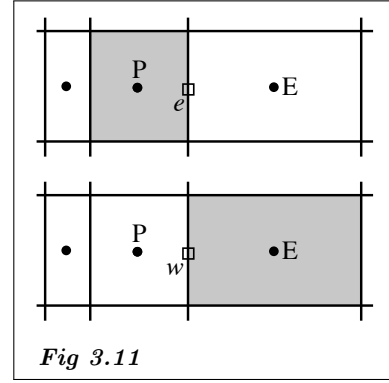
3.6.2 Positive coefficients rule

In a heat conduction problem with no sources ($q = 0$), an increase in the temperature at any grid point should increase the temperature at neighboring points. Suppose the temperature T_P satisfies Eq. (3.22) with $b = 0$,

$$a_P T_P = a_E T_E + a_N T_N + a_W T_W + a_S T_S. \quad (3.39)$$

If T_E is increased by ΔT_E while the other neighbors remain fixed, the change ΔT_P must satisfy

$$a_P (T_P + \Delta T_P) = a_E (T_E + \Delta T_E) + a_N T_N + a_W T_W + a_S T_S. \quad (3.40)$$



Subtracting Eq. (3.39) from Eq. (3.40), we find that

$$a_P \Delta T_P = a_E \Delta T_E \Rightarrow \Delta T_P = \frac{a_E}{a_P} \Delta T_E. \quad (3.41)$$

For a positive ΔT_E to produce a positive ΔT_P , the coefficients a_P and a_E must have the same sign. Repeating this argument for all neighbors shows that all neighbor coefficients a_{nb} must similarly have the same sign. Inspecting Eqs. (3.19)–(3.21), we see that this requires all coefficients to be positive.

3.6.3 Sum of neighbors rule

When $q = 0$, Eq. (3.22) approximates the Laplace equation $\nabla^2 T = 0$. Because the Laplacian involves only derivatives, adding a constant B to T does not change the equation,

$$\nabla^2(T + B) = \nabla^2 T. \quad (3.42)$$

A consistent discrete approximation should preserve this property. Thus

$$a_P T_P = \sum_{nb} a_{nb} T_{nb}, \quad (3.43)$$

$$a_P (T_P + B) = \sum_{nb} a_{nb} (T_{nb} + B). \quad (3.44)$$

Taking the difference of these equations shows that the central coefficient must be the sum of its neighbors,

$$\boxed{a_P = \sum_{nb} a_{nb}}. \quad (3.45)$$

This condition is satisfied by Eq. (3.21) for heat conduction.

3.6.4 Boundedness

The positive coefficients and sum-of-neighbors rules help ensure that finite-volume discretizations of advection and diffusion reproduce the boundedness property discussed in Section 1.4. That is, these operators do not create new maxima or minima. Note, however, that other terms in a PDE, such as source terms, can generate new extrema. For example, the steady solution in Fig. 3.10 has a maximum inside the domain, because a source term was added to force a manufactured solution.

3.6.5 Accuracy

In addition to the principles above, a finite-volume discretization should achieve a reasonable order of accuracy as the grid is refined. This requirement applies both to the numerical approximation of the integral terms and to the associated interpolation and finite-difference schemes used to evaluate fluxes at cell faces.

In practice, most general-purpose finite-volume CFD solvers are designed to maintain second-order accuracy on sufficiently smooth solutions and grids. Achieving this requires consistent use of second-order spatial approximations and careful treatment of geometric and material variations. As shown in the previous section, verification studies play an essential role in confirming that the expected order of accuracy is realized in an actual implementation.

3.6.6 Comparison with finite differences

As a final note, it is instructive to consider the finite-volume stencil (3.18) for the special case of a uniform grid and constant thermal conductivity k . In that case, we can express stencil (3.18) in the form below,

$$k \left(\frac{T_E - 2T_P + T_W}{\Delta x^2} \right) + k \left(\frac{T_N - 2T_P + T_S}{\Delta y^2} \right) + q_P = 0, \quad (3.46)$$

which is exactly the standard second-order finite-difference approximation of the differential form of the diffusion equation,

$$k \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) + q = 0. \quad (3.47)$$

More generally, on a nonuniform grid, the finite-volume method still produces a valid finite-difference approximation of the differential equation (3.47). However, it will be less than second-order accurate, and it will differ from a formally second-order finite-difference approximation. The key advantage of the finite-volume approximation is that it is *flux consistent*. By contrast, if one instead discretizes the differential form directly, using formally second-order accurate finite-difference formulas for $\partial^2 T / \partial x^2$ and $\partial^2 T / \partial y^2$ on a nonuniform grid, the resulting scheme is typically *not* flux consistent.

For the simple steady diffusion problem considered in this chapter, the practical difference between a flux-consistent finite-volume discretization and a formally second-order finite-difference scheme is likely small. The difference is important, however, when approximating the Navier–Stokes equations. In that case, flux-consistent discretizations are typically much more robust and stable.

3.7 Summary of key concepts

Key definitions:

1. **Cartesian grid:** a grid in which all cells are cuboids with faces aligned with the coordinate axes.
2. **Uniform and non-uniform grids:** a grid is uniform when all cells have identical dimensions Δx and Δy , and non-uniform otherwise.
3. **Cell centroid:** the volumetric centroid of a cell.
4. **Face centroid:** the area center of a cell face, where fluxes and face values of field quantities are evaluated.
5. **Ghost node:** an artificial grid point introduced outside the domain to facilitate the application of boundary conditions while preserving the standard interior stencil.
6. **Sparse matrix:** a matrix in which most entries are zero.
7. **Method of manufactured solutions:** a verification technique in which forcing terms are used to force an artificial analytical solution.

Key concepts:

1. Finite-volume methods discretize the integral form of conservation equations over a set of non-overlapping control volumes, enforcing discrete conservation on each cell.
2. Finite-volume methods discretize conservation laws in two steps: integrals are approximated using quadrature rules, and the resulting face values are evaluated using interpolation and finite differences.
3. In second-order accurate finite-volume methods, volume and surface integrals are approximated using the midpoint rule.
4. The face-normal gradients in the diffusion term are typically approximated using centered differences. These are formally second-order accurate on uniform grids, and typically nearly second-order accurate on slowly varying non-uniform grids.
5. The face values of the thermal conductivity are typically obtained by linear interpolation when the conductivity varies smoothly. The harmonic mean is often preferred when properties vary sharply between cells.
6. In this textbook, we apply boundary conditions using ghost nodes. It is important to remember that ghost nodes lie outside the physical domain, and the field values stored there are not generally physically meaningful.
7. A good finite-volume discretization should satisfy three properties: discrete conservation (consistent flux approximations), positive coefficients, and the sum-of-neighbors rule. Together, these ensure that the numerical solution is bounded and physically meaningful.

8. The global matrix system $\mathbf{AT} = \mathbf{b}$ is sparse, and should be stored and solved using sparse linear algebra.
9. Verification using the method of manufactured solutions confirms that a solver is correctly implemented and achieves the expected order-of-accuracy. It should be distinguished from validation, which assesses whether the mathematical model accurately represents the physical problem.
10. For the scope of this introductory textbook, we solve sparse linear systems using direct solvers. In practice, however, 3D finite-volume simulations overwhelmingly rely on iterative solvers.